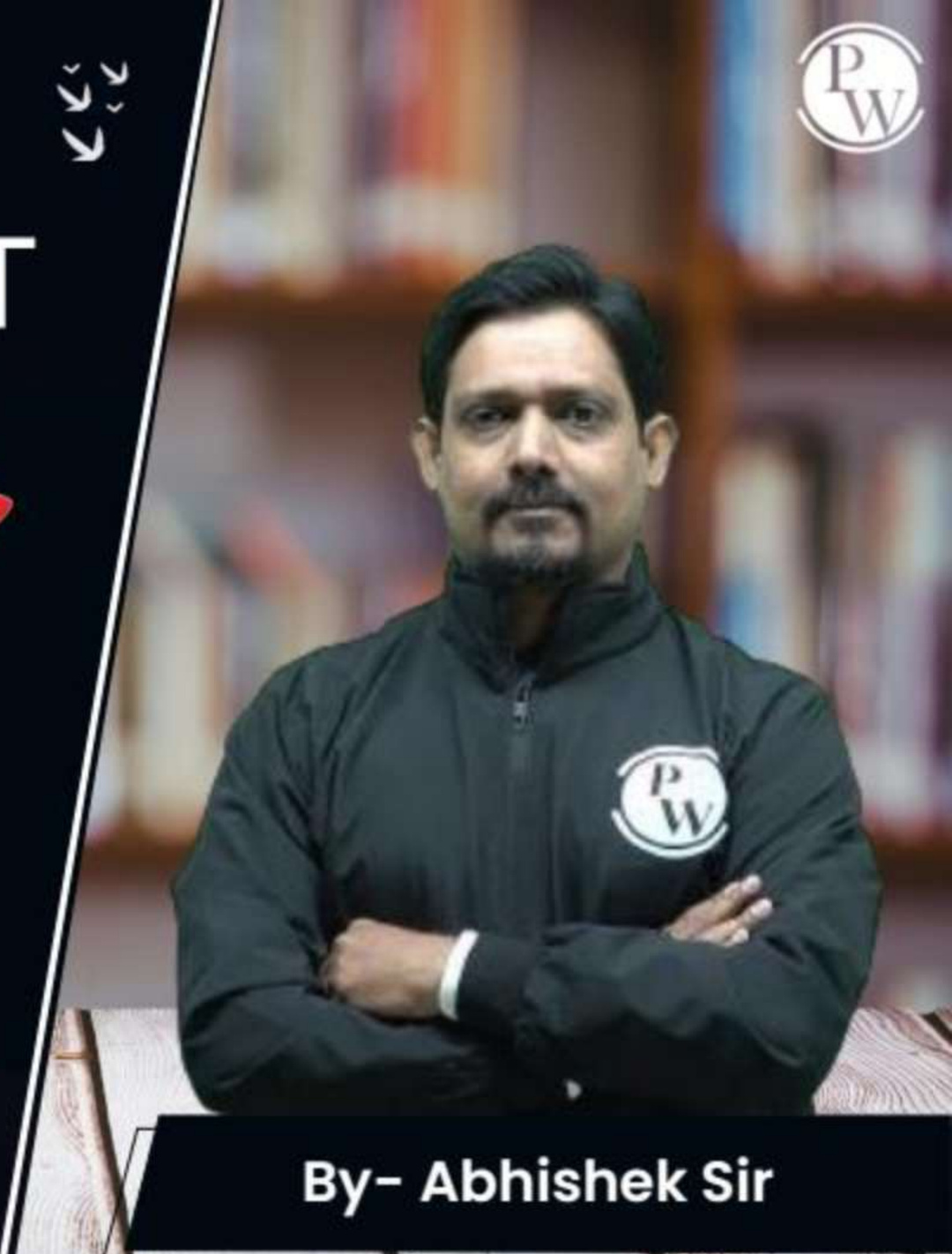
Computer Science & IT



C Programming

Array & Pointers

Lecture No. 04



By- Abhishek Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic
- Topic
- Topic

pointer Arithmetic

$A[i] = *(A+i)$

difference between 2 pointer: $\frac{pto_2 - pto_1}{Size} = pto_2 - pto_1$

L-value R-value

Address $a = 10$ | $b = a$ ← R-value

value

Slide

Topics to be Covered



Topic — Data type Associated 1-D array

Topic — 2-D array Declaration

Topic — A[i][j] - expansion

Topic — 2-D array data type

Topic

Slide

Array

```c
#include <stdio.h>
int main(){
    int a[] = {1,2,3,4};

    printf("%u\n", a);
    printf("%d\n", *a);
    printf("%u\n", &a);

    printf("%u\n", a+1);
    printf("%d\n", *a+1);
    printf("%u\n", &a+1);

    return 0;
}
```

%p

4 element = 4×4B = 16B

A[0]	A[1]	A[2]	A[3]
1	2	3	4
100	104	108	112

Name of array is address of first element

Address of integer : 100 (Assumed)

a+1 = a+1×4 = 100+4 = 104

Slide



Array

```
#include <stdio.h>
int main(){
    int a[] = {1,2,3,4};

    printf("%u\n", a);
    printf("%d\n", *a);
    printf("%u\n", &a);

    printf("%u\n", a+1);
    printf("%d\n", *a+1);
    printf("%u\n", &a+1);

    return 0;
}
```

a[0]	a[1]	a[2]	a[3]
1	2	3	4
100	104	108	112

*(100) = 1

*a+1 = 2

*a++

*(a++) → Error first post operator prefix operator

Slide



Array

```
#include <stdio.h>
int main(){
    int a[] = {1,2,3,4};

    printf("%u\n", a);
    printf("%d\n", *a);
    printf("%u\n", &a);

    printf("%u\n", a+1);
    printf("%d\n", *a+1);
    printf("%u\n", &a+1);

    return 0;
}
```

Slide

Array

A[0]	A[1]	A[2]	A[3]
1	2	3	4
100	104	108	112

a - Address of 1-D array

Address of integer : 100

Address of 1-D array : 100

a → 100

a+1 = 100+1 = 100+1×Size of array = 100+16 = 116



Array

```
#include <stdio.h>
int main(){
    int a[] = {1,2,3,4};

    printf("%u\n", a);
    printf("%d\n", *a);
    printf("%u\n", &a);

    printf("%u\n", a+1);
    printf("%d\n", *a+1);
    printf("%u\n", &a+1);

    return 0;
}
```

A[0]	A[1]	A[2]	A[3]
11	12	13	14
100	104	108	112

Address of integer

integer

Address of 1-D array

Slide

integer Data type and 1-D array data type is different



Array

A[0]	A[1]	A[2]	A[3]
11	12	13	14
100	104	108	112

"%p"

Hexadecimal Address

$b_1 - b = 116 - 100$

$= 16$

$= \frac{16}{4} = 4$

$b - b_1 = \frac{100 - 116}{4} = -4$ %d

*(2a+1)

*(100+1)

*(116) · out of range garbage

Slide

$*(\&a)$ = a - Address of integer

2 D Array

```
int a[2][2] = {{1,2},{3,4}};
```

- `a[2][2]`: `2` → row, `2` → column
- `{{1,2},{3,4}}`: `{1,2}` → 1-D array
- Column-0
- Column-1

array is a Sequential arrangment

	Column-0	Column-1	
0	1	2	← row-0
1	3	4	← row-1

Slide

2D Array

```
int a[3][3] = {1,2,3,4,5,6,7,8,9};
```

No. of 1-D array

Columns 1-D array size

Column-0	Column-1	Column-2	
1	2	3	← row-0
4	5	6	← row-1
7	8	9	← row-2

Slide

2 D Array

```
int a[3][3] = {1,2,3,4,5,6,7,8,9};
```

```
printf("%d", a[1][2]);
```

this will print 6

```
printf("%d", a[0][1]);
```

this will print : 2

Column-0	Column-1	Column 2	
1	(2)	3	← row-0
4	5	(6)	← row-1
7	8	9	← row-2

Slide



2D Array

Raw major-ordering

```
int a[3][3] = {1,2,3,4,5,6,7,8,9};
```

Array stores element in sequential manner

Raw-0			Raw-1			Raw-2		
1	2	3	4	5	6	7	8	9
100	104	108	112	116	120	124	128	132

Column-0 ↓	Column-1	Column-2	
1	(2)	3	← raw-0
4	5	(6)	← raw-1
7	8	9	← raw-2

Raw major : arranging raws one after another

Slide



2 D Array

$a[i] = *(a+i)$

$a[i][j] = *(a+i)[j]$

$= *(*(a+i)+j)$

To access the integer we need 2 time dereferencing

double pointer is integer

Slide



2 D Array

Imp

$a[i][j] = *(a+i)[j]$

$*(*(a+i)+j)$

Add of 1D array

Slide



2D Array

Data type 3 data types

Value will Not change

a → Name of array is Address of first element of array.

a → Address of 1-D array

*a → Address of integer

**a → integer

&a

2D Array

int a[3][3] = {1,2,3,4,5,6,7,8,9};

→ 2D array Size = 9×4 = 36B

→ 1D array = 3×4 = 12B

Array stores element in sequential manner

Row-0			Row-1			Row-2		
1	2	3	4	5	6	7	8	9
100	104	108	112	116	120	124	128	132

a[2][1] =

((a+2)+1)

((100+2)+1)

((100+2×12)+1)

((124)+1)

*(124+1) = *(124+1×4) = *(128) = 8

Slide

2-D Array

```c
#include <stdio.h>

int main(){
    int a[2][2] = {{11,12},{13,14}};

    printf("\n %u",a);

    printf("\n %u",*a);
    printf("\n %u",**a);

    printf("\n %u",a+1);
    printf("\n %u",*a+1);
    printf("\n %u",**a+1);

    return 0;
}
```

11	12	13	14
100	104	108	112

2-D Size = 4 × 4 = 16 B

1-D Size = 2 × 4 = 8B



2-D Array

```
#include <stdio.h>

int main(){
    int a[2][2] = {{11,12},{13,14}};

    printf("\n %u",a);

    printf("\n %u",*a);
    printf("\n %u",**a);

    printf("\n %u",a+1);
    printf("\n %u",*a+1);
    printf("\n %u",**a+1);

    return 0;
}
```

Slide

Array Niwas

{11	12}	{13	14}
100	104	108	112

→ address of 1-D array - 100

→ 100+1 = 100+1×Size

= 100+1×8 = 108



2-D Array

```
#include <stdio.h>

int main(){
    int a[2][2] = {{11,12},{13,14}};

    printf("\n %u",a);

    printf("\n %u",*a);
    printf("\n %u",**a);

    printf("\n %u",a+1);
    printf("\n %u",*a+1);
    printf("\n %u",**a+1);

    return 0;
}
```

11	12	13	14
100	104	108	112

Addrss of integer

→ 100

→ 100+1 = 100+1 x Size
= 100+1x4 = 104

Slide

2-D Array

```c
#include <stdio.h>

int main(){
    int a[2][2] = {{11,12},{13,14}};

    printf("\n %u",a);

    printf("\n %u",*a);
    printf("\n %u",**a);

    printf("\n %u",a+1);
    printf("\n %u",*a+1);
    printf("\n %u",**a+1);

    return 0;
}
```

11	12	13	14
100	104	108	112

**a : a[0][0]

→ 11

*a+1 : 12

Slide



Question

Consider the following C program.

```
# include <stdio.h>
int main (){
int a[4][5] =  {{1, 2, 3, 4, 5},
                {6, 7, 8, 9, 10},
                {11, 12, 13, 14, 15},
                {16, 17, 18, 19, 20}};

printf("%d\n", *(*(a+**a+ 1)+4));
return (0);
}
```

The output of the program is _____.

(A) 5
(B) 10
(C) 15
(D) 20

9pm

Data type

0 → 1 → 2 → 4th

((a+**a+1)+4)

((a+1+1)+4)

((a+2)+4) = a[2][4]

Slide



2 mins Summary

Topic — 1-D array data type

Topic — 2-D array

Topic — 2-D array data type

Topic — a[i][j] = *(*(a+i)+j)

Topic

Slide

THANK - YOU

